Software Test Plan (STP): Online Quiz System

Project: Online Quiz System (console application in C)

Version: 1.0

Authors: Team Jackfruit (Tejas, Sanjana, Vikas, Vaishnavi)

Date: 12-10-2026

Status: Baseline; test cases designed and first results recorded against `main`

1. Introduction

Purpose. This document says how the Online Quiz System is tested: what is tested, how, with what, when and by whom. It links every test case to the SRS requirement it checks.

Scope. All functional requirements REQ-1 to REQ-22 and all non-functional requirements NFR-1 to NFR-5, including the security requirements SEC-1 to SEC-6 of SRS v1.1. Testing is done on the compiled program and on single C modules.

References. SRS v1.1 (`SRS.md`); SAD v1.0 (`docs/SAD.md`); Mini-project deliverables, Part-1 and Part-2.

2. Test Items

- Menu Controller (`src/main.c`) — Tejas
- Quiz Engine (`src/quiz.c` , `data/questions.txt`) — Sanjana
- Scoring Module (`src/score.c`) — Vikas
- Result Storage and Leaderboard (`src/storage.c`) — Vaishnavi
- The integrated program (`quiz` / `quiz.exe`) built by the `Makefile` and GitHub Actions

3. Features to be Tested

SRS ID	Feature	Type
REQ-1 to REQ-6	Main menu: display, validate input, re-prompt, navigate, return, exit	Functional / negative
REQ-7 to REQ-11	Scoring: compare answers, counts, percentage, Pass/Fail at 40%, show result	Functional
REQ-12, REQ-13	Pass the result to storage; save name and score	Integration / functional
REQ-14 to REQ-16	Read results, sorted leaderboard, missing file handled	Functional / negative
REQ-17 to REQ-22	Quiz engine: load, show one question, validate, record, next, finish	Functional
NFR-1	Input processed within 1 second	Performance
NFR-2 (SEC-1 to SEC-6)	Security	Security
NFR-3	Clear prompts and `Error:` messages	Usability
NFR-4	No crash on bad input or missing files	Negative
NFR-5	Modular structure	Inspection

4. Features Not to be Tested

- Login, accounts and roles; network, web, database and GUI (out of scope in the SRS).
- Several users at the same time (SRS constraint: single user).
- The C standard library and the operating system (assumed correct).

- Leaderboards with more than 100 stored results.

5. Test Approach / Strategy

Levels

- **Unit:** one module on its own, using `assert` in `tests/test_score.c` (run with `make test`).
- **Integration:** the Menu Controller with the other modules (quiz, then score, then storage, REQ-12), by running the real program.
- **System:** the whole program checked against the SRS, from the console.
- **Acceptance:** the demo videos at the end of each sprint.

Types. Functional, negative (wrong input and file errors), boundary (40% pass mark), performance (timing), usability and inspection, security (5.1), and regression (earlier tests run again after each merge).

Method. Input is sent to the program from a script (`printf '1\nAsha\n...' | ./quiz`), so each run is the same. The output is compared with what the SRS says.

Entry criteria. The code builds without warnings and the GitHub Actions build is green.

Exit criteria. All planned test cases are run, each failure has a recorded defect with an owner, and no critical defect (crash, data loss, wrong score) is open.

5.1 Security Validation

Security testing covers the six security requirements in SRS Section 5.3.2. The program has no network and no login, so the checks focus on input handling, file handling and not showing system details.

Check	Method	Requirement	Test case
Error text shows no paths or system codes	Cause each error and read the output	SEC-1	TC-16
A player name cannot damage or fake a record	Save names containing `,` , CR and LF; look at `results.txt`	SEC-2	TC-09
Long or wrong input does not overflow or crash	A 5000-character line, an empty line, letters instead of a number	SEC-3	TC-03, TC-17
File problems do not crash the program	Make the results file impossible to open, remove it, leave it empty	SEC-4	TC-14, TC-16
File names cannot be chosen by the user; risky C functions are not used	Code review and search for `gets` , `scanf` , `sprintf` , `strcpy`	SEC-5, SEC-3	TC-18
End of input is handled	End the input at the menu and at the name prompt	SEC-6	TC-05

6. Test Environment

Windows 11 (MinGW GCC, PowerShell or Git Bash) and Ubuntu (GitHub Actions); GCC with `-Wall -std=c99` and GNU Make. Test data: `data/questions.txt` (5 questions) and sample `results.txt` files. Tests are run from the repository root.

7. Test Schedule

- **12 Oct 2026:** test cases designed; unit tests and menu tests run.
- **19 to 23 Oct 2026 (Sprint-1):** integration run of menu, quiz, score and save; demo video (DEMO-1).
- **26 to 30 Oct 2026 (Sprint-2):** leaderboard tests, all test cases run again (TEST-1), final report and demo video (DEMO-2).

8. Test Deliverables

Test Plan (this document), test cases and results (Section 13), unit test source `tests/test_score.c`, defect reports (GitHub issues), and the test summary (Section 14).

9. Roles and Responsibilities

- **Sanjana:** QA lead; runs the full suite and logs results (TEST-1).
- **Vikas:** maintains `tests/test_score.c`; unit and boundary tests.
- **Vaishnavi:** storage, leaderboard and security checks.
- **Tejas:** menu tests, CI, review and merge.

10. Risks and Mitigation

- **Leaderboard (Sprint-2) leaves little time to test:** test cases and sample data are prepared in advance.
- **Tests depend on the working directory:** always run from the repository root.
- **Windows (CRLF) and Linux line endings in `results.txt`:** include a CRLF file in the leaderboard tests.

11. Assumptions and Dependencies

- Modules use the function names and arguments in the header files exactly.
- The question file follows the layout in SAD Section 3.6.
- `results.txt` is created by the program; each test starts from a known state.

12. Suspension and Resumption Criteria

Suspend testing if the build on `main` fails, or if one defect blocks more than 30% of the planned cases. **Resume** when the build is green again and the defect is fixed and merged.

13. Test Case Management and Traceability

13.1 Test Cases

Run on 12-Oct-2026 on `main` after the quiz engine was completed, together with the 8 scoring unit tests in `tests/test_score.c`. **Not run** means planned for Sprint-2.

TC	Requirement	Type	Steps	Expected result	Result
TC-01	REQ-1	Functional	Start the program	Menu shows 1 Start Quiz, 2 View Leaderboard, 3 Instructions, 4 Exit	**Pass**
TC-02	REQ-2, REQ-4	Functional	Enter `3` at the menu	Instructions screen is shown	**Pass**
TC-03	REQ-3, NFR-4, SEC-3	Negative	Enter `abc`, `0`, `5`, an empty line, `4 x`	Each gives `Error: please enter a number from 1 to 4.` and the menu again	**Pass**
TC-04	REQ-5	Functional	Enter `3`, then `4`	Menu is shown again after the instructions	**Pass**
TC-05	REQ-6, SEC-6	Functional / negative	(a) Enter `4`. (b) Empty standard input	Goodbye message and clean exit in both cases	**Pass**
TC-06	REQ-7, REQ-8, REQ-9	Unit	`calculate_result` with 3, 1, 5 and 0 of 5 correct	Scores 3, 1, 5, 0; percentages 60, 20, 100, 0; correct + incorrect = total	**Pass**

TC	Requirement	Type	Steps	Expected result	Result
TC-07	REQ-10	Unit (boundary)	4 of 10, then 3 of 10 correct	40% is Pass; 30% is Fail	**Pass**
TC-08	REQ-11	Unit	`show_result` with a result, then with NULL	Shows score, percentage, counts, Pass/Fail; NULL prints an `Error:` line, no crash	**Pass**
TC-09	REQ-13, SEC-2	Functional / security	Save the name `Tara,Bad`	Record is `Tara Bad,<score>` on one line	**Pass**
TC-10	REQ-12	Integration	Complete a quiz as `Asha`	New line `Asha,<score>` is added to `results.txt`	**Pass** (`Asha,3`)
TC-11	REQ-17 to REQ-22	System	Start quiz, answer every question	One question at a time with options A to D; each answer recorded; result shown at the end	**Pass**
TC-12	REQ-19	Negative	During a question enter `E` , an empty line, `1` , then `B`	`Error:` and the same question each time; `B` is accepted	**Pass**
TC-13	REQ-14, REQ-15	Functional	`results.txt` has `Vikas,10` , `Vaishnavi,8` , `Tejas,9` ; View Leaderboard	Order shown is 10, 9, 8	**Pass** (Vikas 10, Tejas 9, Vaishnavi 8)
TC-14	REQ-16, SEC-4	Negative	Delete `results.txt` ; View Leaderboard	"No results yet" message, back to the menu, no crash	**Pass** ("No results yet" for a missing, empty or unreadable file; bad lines skipped)
TC-15	NFR-1	Performance	Time a full scripted run	Each response within 1 second	**Pass** (full run 0.07 s; leaderboard of 100 results 0.11 s)
TC-16	SEC-1, SEC-4	Security	Make `results.txt` a folder and save a result	Only `Error: could not save your result.` , no path or system text	**Pass**
TC-17	SEC-3, NFR-4	Security / negative	A 5000-character line at the menu, then `4`	One `Error:` message, program continues and exits normally	**Pass**
TC-18	SEC-5, SEC-3	Inspection	Search the source for `fopen` , `gets` , `scanf` , `sprintf` , `strcpy`	File names are fixed constants; no unsafe input functions	**Pass**
TC-19	NFR-5	Inspection	Build `score.c` with its test, without the other modules	Builds and runs on its own	**Pass** (`make test`)
TC-20	NFR-3	Inspection / usability	List every message printed on an error path, then finish a quiz using only the on-screen prompts	Each error starts with `Error:` and is followed by a prompt or the menu; every screen has a clear prompt	**Pass** (5 `Error:` messages; quiz finished with prompts only)

13.2 Requirements Traceability Matrix

Requirement	Test case(s)
REQ-1 to REQ-6	TC-01, TC-02, TC-03, TC-04, TC-05
REQ-7 to REQ-10	TC-06, TC-07

Requirement	Test case(s)
REQ-11	TC-08
REQ-12, REQ-13	TC-10, TC-09
REQ-14 to REQ-16	TC-13, TC-14
REQ-17 to REQ-22	TC-11, TC-12
NFR-1	TC-15
NFR-2 (SEC-1 to SEC-6)	TC-16, TC-09, TC-03, TC-17, TC-14, TC-18, TC-05
NFR-3	TC-20
NFR-4	TC-03, TC-17
NFR-5	TC-19

Every requirement in SRS v1.1 appears in at least one test case.

14. Test Metrics and Reporting

Report after the second execution (27-Oct-2026, with the leaderboard)

- Test cases designed: 20. Executed: 20. **Pass: 20. Fail: 0. Not run: 0.**
- Requirement coverage by test cases: 100% of REQ-1 to REQ-22 and NFR-1 to NFR-5.
- Unit test coverage of `score.c`: 100% of lines and 100% of branches (measured with `gcov`).

A status update is made after each merged pull request that changes how the program behaves, and a final test summary is written at the code freeze.